How the Astronomy Pipeline Fits Together

ECE 379K · HST Team · Oct 21, 2026 · Companion to the project checklist

The goal in one paragraph

The project turns raw telescope images (FITS files) into a catalog: a table listing every bright object found, with its position, size, and brightness. Surveys produce thousands of images a night, so finding objects by eye does not scale. Our pipeline does it automatically, correctly, and quickly.

Success has two measurable parts:

- **Correct.** On synthetic images where we know the true position, size, and brightness of every object, the pipeline recovers them closely, with few missed objects and few false ones.
- **Fast.** On the same batches of images, the CPU+GPU version runs at least 2x faster than a CPU-only version, with no loss of accuracy.

Every checklist item does one of three jobs: it **produces answers** (the pipeline stages), it **proves the answers are right** (synthetic data, validation, tests), or it **makes the pipeline faster** (tiling, workers, GPU, benchmarks). When an item seems pointless, ask which of the three it serves.

The system end to end

One image travels through seven stages, in this order:

#	Stage	Code	Owner	Goes in → comes out	Why it exists
1	Ingest	io_fits.read_fits	Person 1	FITS file → 2D float32 array + header	Everything downstream needs a plain, clean array, not a file format.

#	Stage	Code	Owner	Goes in → comes out	Why it exists
2	Background	background.estimate_background	Person 3	Image → background map + noise level	Sky glow is not uniform. It must be removed so faint objects stand out, and the noise level defines what counts as bright enough.
3	Tile	tiling.make_tiles	Person 2	Image → overlapping tiles	Small independent pieces can run in parallel and be sent to the GPU in batches.
4	Filter	filtering.smooth	Person 3	Tile → smoothed tile	Smoothing averages out pixel noise so real objects stand out. This is the heaviest arithmetic and the GPU target.
5	Detect	detection.detect	Person 3	Smoothed tile → list of sources	Threshold, group connected pixels, and measure each group.
6	Merge	tiling.merge	Person 2	Per-tile lists → one list per image	Removes duplicates at tile borders and numbers the sources.
7	Write	io_fits.write_catalog	Person 1	Sources → CSV or FITS table	This is the deliverable.

Three support systems sit around that main path:

- **Synthetic data and validation (Person 1)** prove the pipeline is correct.
- **The worker pool and run_batch (Person 2)** run many images and tiles at once.
- **The benchmark harness and GPU port (Persons 2 and 3)** measure and deliver the speedup.

The CPU/GPU split follows the nature of the work. The CPU reads files, organizes work, and runs the irregular steps (labeling connected pixels, measuring objects). The GPU runs the filter, which is the same arithmetic applied to every pixel, exactly what GPUs do well.

Key ideas behind the stages

- **Background and noise.** An image is real signal (stars, galaxies) plus a slowly varying sky glow plus random noise in every pixel. The background map captures the glow; the noise level (sigma) measures the random scatter.
- **Smoothing.** Blurring the image slightly averages neighboring pixels. Random noise partly cancels while a real object, which spans several pixels, survives. This makes faint objects detectable.
- **Threshold.** A pixel counts as part of an object only if it is more than k times sigma above background. A small k finds more faint objects but also more false ones; a large k does the reverse.
- **Tile and halo.** A tile is a rectangular piece of the image. Its core is the part it is responsible for; its halo is extra border pixels copied from neighbors. Filtering needs neighboring pixels, so without a halo the results at a tile edge would differ from whole-image processing.
- **Ownership rule.** An object near a tile border may be visible to two tiles. Only the tile whose core contains the object's center reports it, so nothing is counted twice or lost.
- **CPU reference.** The CPU filter is the trusted answer. The GPU version is correct only if it matches it numerically.

The checklist, item by item

Each item below says what it is, why it exists in the design, and how you know it is done.

Everyone (Oct 20-22)

Agree on interfaces.py. *What:* all three sign off on the shared data types, coordinate conventions, and function signatures. *Why:* you write code at the same time without seeing each other's work. If one person returns tile-local coordinates and another expects full-image coordinates, the bug only appears at integration, when it is expensive. *Done when:* all three have read the conventions and agree.

Shared repo with a tests folder. *Why:* one source of truth, and tests let anyone check that a change did not break someone else's piece. *Done when:* everyone can clone, run the tests, and push.

Request the GPU allocation. *Why:* GPU time is granted through TACC, and approvals and queue waits take days. Asking now means November GPU work is not blocked. *Done when:* the request is submitted and you know the queue names.

Confirm the Python environment on TACC. *Why:* the GPU plan depends on CuPy working on the cluster. Finding a missing or incompatible package now is far cheaper than finding it on Nov 4. *Done when:* astropy, NumPy, and SciPy import on a compute node, and you know how CuPy will be installed.

Person 1: ingestion, data, validation

Synthetic image generator, FITS output, and truth table (Oct 20-22). *What:* a program that paints fake stars and galaxies with known positions, sizes, and brightness onto a noisy background, and saves the answer key. *Why:* real telescope images have no answer key, so without this we cannot say whether the pipeline is accurate. It also unblocks Persons 2 and 3, who need images to test on before the real download finishes. *Done when:* the same seed gives the same image, and the truth file matches what was painted.

Three synthetic sets: tiny, medium, large (Oct 22). *Why:* different jobs need different data. Tiny runs in seconds for unit tests. Medium is for tuning the threshold and kernel width and checking accuracy. Large gives enough work that benchmark timings mean something. *Done when:* all three exist in a shared location.

Start the SDSS download (Oct 20-22). *Why:* 500 images (about 6 GB) take time and the download depends on nothing else, so it runs in the background. Real images show what the pipeline will face: messier noise, bad pixels, bright saturated stars. They are also the large batch behind the speed claim. Note that real images have no answer key here, so they test robustness and speed, while synthetic images test accuracy. *Done when:* the files sit in $WORK, which all TACC systems share.

read_fits (Oct 27). *What:* opens a file, finds the image data, converts it to float32, and replaces missing values. *Why:* this is stage 1. Real FITS files have quirks (compression, data stored in an extension, missing pixels), and handling them once here means the rest of the pipeline only ever sees a clean array. *Done when:* it reads both synthetic and SDSS files.

write_catalog (Oct 27). *Why:* this is stage 7. A fixed column schema means validation, plotting, and the report all read the same table. Sorted rows make output deterministic, so two runs can be compared with a simple diff. *Done when:* the output matches the schema in interfaces.py.

score_catalog (Oct 27). *What:* matches detections to the truth table and reports completeness, false-positive rate, position error, and flux error. *Why:* it turns 'looks right' into numbers. These are the accuracy evidence for the report, and the tool that shows whether speed work damaged accuracy. *Done when:* running it on the medium set prints all four numbers.

Integrate ingestion with tiling and the worker pool (Nov 3). *Why:* this is the first time real files flow through the parallel machinery. It catches problems synthetic data hides, such as odd headers, file sizes, and memory use. *Done when:* a batch of real images goes from files to catalog with 2-4 workers.

Review Person 3's detection code (Nov 3). *Why:* detection is the most specialized code in the project and one person knows it. A second reader finds bugs and can take over if a teammate becomes unavailable, which is on the risk list. *Done when:* you can explain what detect() does and have raised any questions.

CPU-vs-GPU comparison script (Nov 12). *Why:* the GPU port is correct only if it matches the CPU output. This script measures the maximum absolute and relative difference at each stage, so you can see where any drift appears. *Done when:* it runs on one tile and reports pass or fail against the tolerance.

Accuracy regression on every merged change (Nov 12 onward). *Why:* optimization is where accuracy quietly breaks. Rerunning score_catalog on the same synthetic set after every change catches it immediately. This protects the 'without reducing accuracy' half of the performance goal. *Done when:* the scores are logged after each merge and compared to the last run.

Full 500-image validation with tables and plots (Nov 19). *Why:* this is the final evidence, the whole system at full scale. *Done when:* the accuracy tables and figures for the report are generated from one command.

Report: data, validation, and results (Nov 24). *Why:* the grade rests on showing the work, and these sections hold the proof.

Person 2: tiling, parallelism, benchmarking

Tiling prototype on random arrays (Oct 22). *Why:* tiling is where subtle bugs live, especially off-by-one errors at edges. Working out the geometry on random data means you are not waiting on anyone else's code.

make_tiles (Oct 27). *What:* cuts an image into tiles with a halo of overlapping border pixels. *Why:* tiles are the unit of parallel work and GPU batching, and the halo keeps results at tile edges equal to whole-image processing. *Done when:* the cores cover every pixel exactly once.

merge with the ownership rule and deterministic source IDs (Oct 27). *Why:* this is stage 6. Objects near borders appear in two tiles' views, and merge keeps each one exactly once. Sorting by position gives the same IDs every run. *Done when:* no object is duplicated

or lost at a border.

Test: tiled result equals whole-image result (Oct 27). *Why:* this defines correct tiling. If tiling changes the answer, there is a bug or the halo is too small. *Done when:* the test passes on the tiny and medium sets.

run_batch with ProcessPoolExecutor (Nov 3). *What:* runs many images at once across CPU cores. *Why:* it makes the CPU side fast and keeps the GPU supplied with work. It uses processes rather than threads because Python's global interpreter lock stops threads from running CPU-heavy code in parallel. Whole-image parallelism comes first because it is simplest and is also your fallback if tiling fails. *Done when:* a batch runs with 2-4 workers.

Test: 1, 2, and 4 workers give identical catalogs (Nov 3). *Why:* parallel bugs such as race conditions make results change from run to run. Identical output proves parallelism changes speed only, never answers.

Measure the I/O share of runtime (Oct 31). *Why:* this tells you whether the 2x goal is realistic. By Amdahl's law, if the filter is a fraction f of total time and the GPU speeds it up by a factor S, overall speedup is 1 / ((1 - f) + f / S). Even an infinitely fast GPU (S very large) gives at most 1 / (1 - f). If file reading is 60% of the time, f is at most 0.4 and the best possible speedup is about 1.7x. Knowing this early lets you attack I/O before it is too late.

Decide: keep tiling or fall back to whole images (Oct 31). *Why:* tiling adds complexity and risk. Keep it only if it works and helps. The proposal already names whole-image processing as the fallback.

GPU batching (Nov 12). *Why:* moving data between CPU memory and GPU memory is slow compared with the arithmetic itself. Sending many tiles per transfer and overlapping file reads with computation keeps the GPU busy.

Benchmark harness with per-stage timings (Nov 19). *Why:* the 2x claim needs numbers. Per-stage times show where time goes. The CPU-only and GPU runs must use the same node type and worker count, or the comparison is unfair. *Done when:* one command prints io, background, filter, detect, merge, and total times.

Report: parallelism and performance (Nov 24). *Why:* these sections hold the speed evidence.

Person 3: filtering, detection, GPU

Prototype filter and detector on the tiny set (Oct 22). *Why:* you can start immediately on synthetic data, and the prototype exposes design questions early.

estimate_background (Oct 27). *What:* estimates the sky level on a coarse grid, ignoring bright objects by sigma clipping, then interpolates to a smooth map; it also returns the noise level. *Why:* this is stage 2. Subtracting the background lets one threshold work across the whole image, and the noise level sets how bright 'bright enough' is. It runs on the whole image before tiling so tiles never disagree about the sky level.

smooth, CPU version (Oct 27). *Why:* this is stage 4 and the reference answer the GPU version must match. It uses FFT-based Gaussian smoothing from SciPy.

detect (Oct 27). *What:* thresholds the smoothed tile, groups connected pixels into objects, and measures each one's centroid, bounding box, size, and brightness in full-image coordinates. *Why:* this is stage 5, where pixels become scientific results. It returns only objects whose centers lie in the tile's core so merge can stay simple.

Integrate detection into Person 2's pipeline (Nov 3). *Why:* detection must work on tiles and inside worker processes, not just on one array. This is where interface mismatches show up.

Tune threshold and kernel width (Nov 3). *Why:* two settings trade completeness against false positives, and the synthetic truth tells you which values work best. One design question to settle here: smoothing lowers the noise, so decide whether the threshold uses the raw noise level or the noise of the smoothed image.

Freeze the CPU reference (Nov 3). *Why:* the GPU port needs a fixed target. If the CPU version keeps changing, you cannot tell whether a mismatch is a GPU bug or a moved goalpost.

smooth, GPU version with CuPy (Nov 12). *Why:* this delivers the speed objective. FFT filtering is the same math at every pixel, which suits the GPU. *Done when:* output matches the CPU reference within about 1e-4 relative error.

GPU go/no-go (Nov 10). *Why:* a time box. If the GPU path is not close to matching by Nov 10, you fall back to the CPU filter and still deliver a working pipeline, as the proposal's reduced scope describes.

Profile and optimize the GPU path (Nov 19). *Why:* the FFT itself is usually fast; time often goes to copying data and launching work. Profiling finds the real bottleneck instead of guessing.

Report: algorithm and GPU (Nov 24). *Why:* these sections explain how detection works and how the GPU speedup was achieved.

Final week (Nov 20-24)

Document limitations. *Why:* the proposal promises it, and honest limits (tile boundary effects, crowded fields, threshold sensitivity, speedup capped by I/O) make the report credible.

Combine the report and check figures. *Why:* sections written separately need one consistent voice, numbering, and set of numbers.

Re-run the full pipeline from a clean checkout. *Why:* it proves the results are reproducible and do not depend on files that exist only on one person's account.

Submit.

Risk watch

Oct 31: is I/O dominating runtime? Answers whether the 2x target is realistic (see Amdahl's law above). **Nov 4: GPU allocation granted and CuPy imports on a GPU node.** Without this, the GPU milestone cannot start. **Nov 10: GPU match go/no-go.** Keeps the project from burning its last weeks on a port that is not converging.

Who waits on whom

Needs this	From	By	Needed for
Agreed interfaces	Everyone	Oct 22	All three writing code in parallel
Synthetic images and truth	Person 1	Oct 22	Everyone's first tests
read_fits, make_tiles, merge, background, detect	Persons 1, 2, 3	Oct 27	First end-to-end baseline
Working process_image	Person 3	Oct 27	Person 2's run_batch
Frozen CPU reference	Person 3	Nov 3	GPU port and comparison script
GPU filter that matches	Person 3	Nov 12	Full benchmark

Needs this	From	By	Needed for
Benchmark and validation results	Persons 1, 2	Nov 19	The report

Why the milestones are in this order

The plan builds a correct but slow version first, then makes it fast. You never optimize something you cannot verify: the synthetic data and scoring tools come first so every later change can be checked. The baseline (Oct 27) proves the stages fit together. Parallelism (Nov 3) adds speed on the CPU. The GPU (Nov 12) adds the final speedup, with the frozen CPU reference as the safety net. The last week is for evidence and writing, not new features.